

✓ Lab 5 Report:

Create Arthur Conan Doyle AI with RNN

✓ Name: Josh Lando

```
%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt
import torch
import wandb
from torch.distributions import Categorical
```

```
from IPython.display import Image # For displaying images in colab ju
```

```
device = torch.device("mps" if torch.mps.is_available() else "cpu")
print(f"Using device: {device}")
```

Using device: cpu

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

Mounted at /content/drive

✓ Prepare Data

```
# First n-characters to use for training
data_size_to_train = 10000

# Load the Shakespeare data up to data_size_to_train
data = open('/content/drive/MyDrive/Colab Notebooks/sherlock.txt', 'r')

# Find the unique characters within the training data
characters = sorted(list(set(data)))

# total number of characters in the training data and number of unique
data_size, vocab_size = len(data), len(characters)

print("Data has {} characters, {} unique".format(data_size, vocab_size))
```

```
Data has 10000 characters, 65 unique
```

```
# Use Python Dictionary to map the characters to numbers and vice versa

character_to_num = { ch:i for i,ch in enumerate(characters) }
num_to_character = { i:ch for i,ch in enumerate(characters) }

print(character_to_num)
```

```
{'\n': 0, ' ': 1, '!': 2, '"': 3, "'": 4, '(': 5, ')': 6, ',': 7, '-': 8, ':': 9, ';': 10, '<': 11, '>': 12, '=': 13, '?': 14, '@': 15, '[': 16, ']': 17, '^': 18, '_': 19, '`': 20, '{': 21, '|': 22, '}': 23, '~': 24, 'a': 25, 'b': 26, 'c': 27, 'd': 28, 'e': 29, 'f': 30, 'g': 31, 'h': 32, 'i': 33, 'j': 34, 'k': 35, 'l': 36, 'm': 37, 'n': 38, 'o': 39, 'p': 40, 'q': 41, 'r': 42, 's': 43, 't': 44, 'u': 45, 'v': 46, 'w': 47, 'x': 48, 'y': 49, 'z': 50, 'A': 51, 'B': 52, 'C': 53, 'D': 54, 'E': 55, 'F': 56, 'G': 57, 'H': 58, 'I': 59, 'J': 60, 'K': 61, 'L': 62, 'M': 63, 'N': 64, 'O': 65}
```

```
# Use the character_to_num dictionary to map each character in the training data to a number

data = list(data)

for i, ch in enumerate(data):
    data[i] = character_to_num[ch]

print(data[:10])
```

```
[0, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

✓ Define Sweep

```
import pprint
```

```
sweep_config = {
    'method': 'bayes'
}

metric = {
    'name': 'val_loss',
    'goal': 'minimize'
}

sweep_config['metric'] = metric

parameters_dict = {
    'optimizer': {
        'value': 'adamw'
    },
    'learning_rate': {
        'distribution': 'uniform',
        'min': 1e-4,
        'max': 1e-2
    },
    'epochs': {
        'value': 3
    },
    'training_sequence_length': {
        'value': 512
    },
    'fc_layer_size': {
        'value': 512
    },
    'num_layers': {
        'value': 3
    },
    'dropout': {
        'values': [0, 0.1, 0.3, 0.4, 0.5, 0.6, 0.7]
    },
    'weight_decay': {
        'distribution': 'uniform',
        'min': 0,
        'max': 0.3
    }
}

sweep_config['parameters'] = parameters_dict

pprint.pprint(sweep_config)
```

```
{'method': 'bayes',  
 'metric': {'goal': 'minimize', 'name': 'val_loss'},  
 'parameters': {'dropout': {'values': [0, 0.1, 0.3, 0.4, 0.5, 0.6, 0.7],  
                             'epochs': {'value': 3},  
                             'fc_layer_size': {'value': 512},  
                             'learning_rate': {'distribution': 'uniform',  
                                                'max': 0.01,  
                                                'min': 0.0001},  
                             'num_layers': {'value': 3},  
                             'optimizer': {'value': 'adamw'},  
                             'training_sequence_length': {'value': 512},  
                             'weight_decay': {'distribution': 'uniform',  
                                                'max': 0.3,  
                                                'min': 0}}}}
```

```
sweep_id = wandb.sweep(sweep_config, project="phys417-lab5")
```

Create sweep with ID: nv54sn3d

Sweep URL: <https://wandb.ai/b6k4pq9yqk-university-of-washington/phys417>

✓ Define Model

```

class CharRNN(torch.nn.Module):

    def __init__(self, num_embeddings, embedding_dim, input_size, hidden_size, dropout):
        super(CharRNN, self).__init__()

        # Define embedding layer
        # num_embeddings = number of unique characters
        # embedding_dim = size of a vector that encodes each character
        self.embedding = torch.nn.Embedding(num_embeddings, embedding_dim)

        # Define vanilla RNN cell
        # num_layers = Number of RNN cells to be stacked for deep-RNN
        # nonlinearity = Activation function to use
        self.rnn = torch.nn.RNN(input_size=input_size, hidden_size=hidden_size,
                                num_layers=num_layers,
                                nonlinearity = 'tanh',
                                dropout=dropout)

        # decoder layer that takes hidden states as inputs and output
        # output_size = number of unique characters
        self.decoder = torch.nn.Linear(hidden_size, output_size)

    def forward(self, input_seq, hidden_state):

        # Forward pass input sequence to embedding layer
        embedding = self.embedding(input_seq)

        # RNN cell takes output of embedding layer + previous hidden state as input
        output, hidden_state = self.rnn(embedding, hidden_state)

        # Forward pass the RNN cell output to decoder to get the probability
        output = self.decoder(output)

        # hidden states need to be detached from computation graph to return output
        return output, hidden_state.detach()

```

✓ Define Hyperparameters

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Fix random seed
torch.manual_seed(25)

# Derfine model size
embedding_dim = 100
input_size = 100

def train(config=None):
    # Initialize a new wandb run
    with wandb.init(config=config):
        # If called by wandb.agent, as below,
        # this config will be set by Sweep Controller
        config = wandb.config

        rnn = CharRNN(num_embeddings = vocab_size, embedding_dim = eml
                      input_size = input_size, hidden_size = config.fc_layer_
                      output_size = vocab_size, dropout = config.dropout)

        rnn.to(device)

        optimizer = build_optimizer(rnn, config.optimizer, config.lear

        loss_fn = torch.nn.CrossEntropyLoss()

        # Fetch global data
        global data
        # Safely extract list whether it is a tensor or already a list
        data_list = data.tolist() if isinstance(data, torch.Tensor) e

        # Convert training data into torch tensor and make it into vec
        tensor_data = torch.unsqueeze(torch.tensor(data_list), dim =

        for epoch in range(config.epochs):
            train_epoch(rnn, optimizer, config.training_sequence_leng
            avg_val_loss = validate_epoch(rnn, loss_fn, tensor_data)
            wandb.log({"val_loss": avg_val_loss, "epoch": epoch})

```

```
def build_optimizer(network, optimizer, learning_rate, weight_decay):
    if optimizer == "sgd":
        optimizer = torch.optim.SGD(network.parameters(),
                                      lr=learning_rate, momentum=0.9, weight_decay=weight_decay)
    elif optimizer == "adamw":
        optimizer = torch.optim.AdamW(network.parameters(),
                                       lr=learning_rate, weight_decay=weight_decay)
    return optimizer
```

✓ Train Model

```
def train_epoch(network, optimizer, training_sequence_len, loss_fn, data_loader):
    # Set the network to training mode
    network.train()

    # Randomly select a starting character from first 100 characters :
    character_loc = np.random.randint(100)

    # initialize initial hiddens state as None
    hidden_state = None

    while character_loc + training_sequence_len + 1 < max_len: # loop

        # Define input/target sequence
        input_seq = data[character_loc : character_loc + training_sequence_len]
        target_seq = data[character_loc + 1 : character_loc + training_sequence_len]

        # Pass input sequence and hidden_state to RNN
        output, hidden_state = network(input_seq, hidden_state)

        # Compute loss between RNN output sequence vs target sequence
        # torch.squeeze removes the column dimension and make them in 1D
        loss = loss_fn(torch.squeeze(output), torch.squeeze(target_seq))

        # wandb.log({"train loss": loss.item()})

        # Empty gradient buffer -> backpropagation -> update network
        optimizer.zero_grad()
        loss.backward()

        # Prevent exploding gradients by clipping the gradients
```

```

torch.nn.utils.clip_grad_norm_(network.parameters(), max_norm=1.0)

optimizer.step()

# Update starting character for next sequence
character_loc += training_sequence_len

def validate_epoch(network, loss_fn, data, max_len = 10000, validation_sequence_len = 100):
    # Set the network to evaluation mode
    network.eval()

    # Randomly select a starting character from first 100 characters
    character_loc = np.random.randint(100)

    # initialize initial hiddens state as None
    hidden_state = None
    epoch_validation_losses = [] # To collect losses for this validation epoch

    with torch.no_grad(): # Disable gradient calculations for validation
        while character_loc + validation_sequence_len + 1 < max_len:

            # Define input/target sequence
            input_seq = data[character_loc : character_loc + validation_sequence_len]
            target_seq = data[character_loc + 1 : character_loc + validation_sequence_len + 1]

            # Pass input sequence and hidden_state to RNN
            output, hidden_state = network(input_seq, hidden_state)

            # Compute loss between RNN output sequence vs target sequence
            # torch.squeeze removes the column dimension and make them 1D
            loss = loss_fn(torch.squeeze(output), torch.squeeze(target_seq))

            epoch_validation_losses.append(loss.item())

            # Update starting character for next sequence
            character_loc += validation_sequence_len

    return np.mean(epoch_validation_losses) if epoch_validation_losses else None

```



```
wandb.agent(sweep_id, train, count=100)
```

[Show hidden output](#)

✓ Train Final

```
config = {
    'optimizer': 'adamw',
    'learning_rate': 0.00069104416484892,
    'epochs': 50,
    'training_sequence_length': 512,
    'fc_layer_size': 512,
    'num_layers': 3,
    'dropout': 0.1,
    'weight_decay': 0.022107379060116103
}

rnn = CharRNN(num_embeddings = vocab_size, embedding_dim = embedding_dim,
              input_size = input_size, hidden_size = config['fc_layer_size'], num_layers = num_layers,
              output_size = vocab_size, dropout = config['dropout'])

rnn.to(device)

optimizer = build_optimizer(rnn, config['optimizer'], config['learning_rate'])

loss_fn = torch.nn.CrossEntropyLoss()

data_list = data.tolist() if isinstance(data, torch.Tensor) else data

# Convert training data into torch tensor and make it into vertical orientation
tensor_data = torch.unsqueeze(torch.tensor(data_list), dim = 1).to(device)

train_loss_list = []

for epoch in range(config['epochs']):
    train_epoch(rnn, optimizer, config['training_sequence_length'], loss_fn, tensor_data)
    avg_val_loss = validate_epoch(rnn, loss_fn, tensor_data)
    train_loss_list.append(avg_val_loss)
    print(f"Epoch {epoch} - Validation Loss: {avg_val_loss:.4f}")
```

[Show hidden output](#)

✓ Visualize & Evaluate Model

```
# Print a validation text sequence that most closely resembles Sherlock
validation_sequence_len = 20000
character_loc = np.random.randint(100)

# initialize initial hiddens state as None
hidden_state = None

# Convert the single input character into a 2D tensor (seq_len=1, batch
input_seq = torch.tensor([[data[character_loc]]]).to(device)

rnn.eval()

with torch.no_grad():

    while character_loc < validation_sequence_len: # Loop continues unt

        # Pass validation sequence to RNN
        # Note that RNN now uses its previous output character as input
        output, hidden_state = rnn(input_seq, hidden_state)

        # Take the softmax of the decoder output to get the probabiliti
        output = torch.nn.functional.softmax(torch.squeeze(output), dim
        # Use the probabilities to sample the output character
        character_distribution = torch.distributions.Categorical(output
        character_num = character_distribution.sample()

        # Convert the character number selected from sampling to actual
        print(num_to_character[character_num.item()], end='')

        # Update the input_seq so that it's using the output of the RNN
        input_seq[0][0] = character_num.item()

        # Update the character location
        character_loc += 1
```

```
Inderirsted to Nad have had already
improved so far as to be able to walk about the wards, and even to
bask a little upon the verandah, when I was struck down by enteric
fever, that curse of our Indicale faz
    The Adventure of the Lion's Mane
    The Adventure of the Retired Colourman
```

it by the time that I communicatter., and we fires Pordleny un than turning round I recognized young Stamford, who had been a dresser under me at Bart's. The sight of a friendly face in the great wilderness of London is a pleasantery thy

The Adventure of the Bruce-Partington Plans

A Case of Identity

The Goleen a dresser

under me at Bart's. The sight of a my Plarlet a paly and to take up in the the problem of I from thin the enemy's country. I followed, however, with many other office

The Adventure Of The Mazarin Stone

The Adventure of the Priory School

Phe Adventure of the Golden Pincend, I was dispatched, accordi undergone, I was removed, with a great train of, and which were too to spend the next nine months in attempting to improve it.

I had neither kith nor kin in England, and was therefore as free as air—or as free as an incomm and quiet habits. I am not strong enough yet to stand much noise or excitement, I was diluat on the sh turning round I recognized young Stamford, who had been a dresser under me at Bart's. The siget alanbet him."

"The Adventure of the eruzery than I ought. Go my.e

The Adventure of the Briechool to London, that great cesspool and idlers of the Empire are irresistibly drained. There I stayed for some time at a private hotel in the Houged ha was thing Scancoms. In queer in his ideas—an enthusiast in some branches of science. As far as I knould like to meet him," I said. "If I ammtor on Prattled have an outser, and

undergone, I was removed, with a great trat Bo table on the sery tom oud dustadation in my style of living. Sherlock Holmes yet," he said care for him as a my hifrtern, and we started off together in a hansom, and he is a first-class chemist; but. In ond in

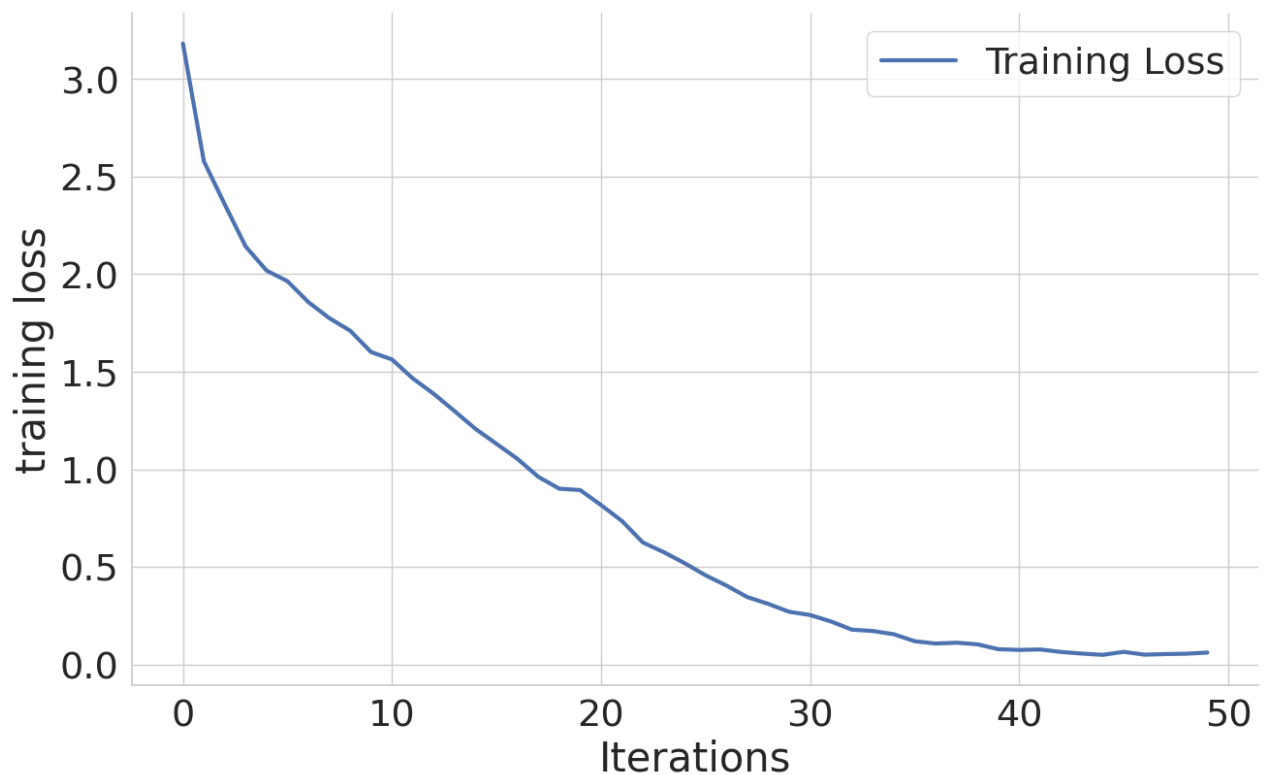
undisguised wonder, as we rattled through the passes, and was already country. I followed, however, with many other officers who were in the same situation as myself, and succeeded in raings.e

The Adventure of the Priory " was removed from my brigade
and attached to the Fifth Northumberland

```
# Import seaborn for prettier plot
import seaborn as sns

sns.set(style = 'whitegrid', font_scale = 2.5)
```

```
# Plot the training loss and rolling mean training loss with respect  
# Feel free to change the window size  
plt.figure(figsize = (15, 9))  
  
plt.plot(train_loss_list, linewidth = 3, label = 'Training Loss')  
# plt.plot(np.convolve(train_loss_list, np.ones(100), 'valid') / 100,  
#          linewidth = 3, label = 'Rolling Averaged Training Loss')  
plt.ylabel("training loss")  
plt.xlabel("Iterations")  
plt.legend()  
sns.despine()
```



Start coding or generate with AI.